

Índice general

1. System Overview	5
1.1. Purpose of the System	5
1.1.1. What the System Does	5
1.1.2. Who Uses the System	5
1.1.3. What Problem the System Solves	6
1.1.4. Bidirectional Bluetooth Communication	6
1.1.5. Real-Time Control and Telemetry Feedback	6
1.1.6. Modular and Expandable Architecture	7
1.2. System Objectives	7
1.2.1. Deterministic Control Loop	7
1.2.2. Low-Latency Command Processing	8
1.2.3. Robust Packet Synchronization	8
1.2.4. Modular Firmware Structure	8
1.2.5. Hardware Abstraction	9
1.2.6. Expandability and Experimentation	9
1.3. High-Level System Description	10
1.3.1. Android Application	10
1.3.2. Bluetooth Communication Layer	10
1.3.3. Protocol Layer	10
1.3.4. Control Processing Layer	10
1.3.5. Telemetry Processing Layer	11
1.3.6. Hardware Abstraction Layer	11
1.3.7. Physical Hardware	11
1.3.8. System Block Diagram	12
1.4. Functional Architecture	12
1.4.1. Downlink: Control Path	13
1.4.2. Uplink: Telemetry Path	13
1.4.3. Bidirectional Operation	14
1.5. Real-Time Characteristics	15
1.5.1. Polling-Based Architecture	15
1.5.2. Non-Blocking Loop Design	15
1.5.3. Avoidance of <code>delay()</code> Usage	15
1.5.4. Deterministic Execution Order	16
1.5.5. Fixed-Size Packet Model for Control	16
1.5.6. Preparation for Timing Analysis	16
1.6. Modular Firmware Philosophy	17
1.6.1. Separation of Responsibilities	17

1.6.2.	Hardware Abstraction Separation	17
1.6.3.	Educational and Practical Benefits	18
1.6.4.	Scalability and Future Extension	18
1.7.	System Capabilities	18
1.7.1.	Motion Control	19
1.7.2.	Digital Output Control	19
1.7.3.	Sensor Integration	19
1.7.4.	Telemetry and Monitoring Features	19
1.7.5.	Platform Flexibility	20
1.8.	System Constraints	20
1.8.1.	Bluetooth Bandwidth and Latency	20
1.8.2.	Microcontroller Resource Limits	20
1.8.3.	Fixed Buffer Sizes	21
1.8.4.	Checksum Simplicity	21
1.8.5.	Single-Loop Execution Model	21
1.8.6.	Hardware Dependency	21
1.8.7.	Educational Scope	22
1.9.	Document Structure	22
2.	Firmware Architecture	24
2.1.	Architectural Objectives	24
2.2.	Architectural Decomposition Model	24
2.2.1.	Logical Layers	24
2.3.	Module-Level Decomposition	25
2.4.	Execution Model	25
2.4.1.	Deterministic Ordering	26
2.4.2.	Polling Strategy	26
2.4.3.	Non-Blocking Constraint	26
2.5.	Data Ownership and State Management	26
2.5.1.	State Variables	26
2.5.2.	State Update Discipline	26
2.6.	Data Flow Structure	26
2.6.1.	Command Processing Path	26
2.6.2.	Telemetry Processing Path	27
2.7.	Interface Boundaries	27
2.8.	Hardware Abstraction Strategy	27
2.9.	Scalability and Controlled Expansion	27
2.9.1.	Sensor Integration	27
2.9.2.	Actuator Integration	28
2.9.3.	I/O Expansion	28
2.9.4.	Structural Stability	28
3.	Communication Protocol Specification	29
3.1.	Protocol Overview	29
3.2.	Communication Model	29
3.3.	Packet Framing Strategy	29
3.3.1.	General Packet Structure	30
3.3.2.	Start-of-Frame Marker	30

3.4.	Control Packet Format	30
3.4.1.	Control Packet Structure	30
3.4.2.	Control Fields	30
3.5.	Telemetry Packet Format	31
3.5.1.	Telemetry Packet Structure	31
3.5.2.	Telemetry Fields	31
3.6.	Checksum and Error Detection	31
3.6.1.	Checksum Purpose	31
3.6.2.	Checksum Model	31
3.7.	Parser Synchronization Model	32
3.8.	Timing Characteristics	32
3.9.	Protocol Determinism	32
3.10.	Extensibility Strategy	32
3.11.	Protocol Robustness Considerations	33
3.12.	Summary	33
4.	Firmware Structure	34
4.1.	Introduction	34
4.2.	Source Code Organization Strategy	34
4.3.	Main Entry Point Structure	34
4.3.1.	Initialization Phase	35
4.3.2.	Main Control Loop	35
4.4.	Module Categories	35
4.4.1.	Communication Module	35
4.4.2.	Control and Input Module	36
4.4.3.	Motor Control Module	36
4.4.4.	Servo Control Module	36
4.4.5.	Telemetry Module	36
4.4.6.	Sensor Interface Modules	36
4.4.7.	GPIO Expansion Driver	37
4.4.8.	Output Management Module	37
4.5.	Inter-Module Communication Model	37
4.6.	Header File Discipline	37
4.7.	Configuration and Pin Isolation	38
4.8.	Error Handling Strategy	38
4.9.	Resource Management Considerations	38
4.10.	Implementation Consistency with Architecture	38
4.11.	Summary	39
5.	Timing and Real-Time Analysis	40
5.1.	Introduction	40
5.2.	Execution Model Overview	40
5.3.	Control Loop Structure	40
5.4.	Worst-Case Execution Time (WCET)	41
5.4.1.	Qualitative Estimation	41
5.5.	Loop Frequency Estimation	41
5.6.	Communication Latency	42
5.7.	Actuator Update Timing	42

5.8. Sensor Sampling Rate	42
5.9. Determinism Analysis	43
5.10. Jitter Considerations	43
5.11. Real-Time Classification	43
5.12. Communication Loss Handling	44
5.13. Scalability Impact on Timing	44
5.14. Summary	44
6. Hardware Architecture and Implementation	45
6.1. Introduction	45
6.2. System Overview	45
6.3. Microcontroller Unit	46
6.4. Motor Driver Stage	46
6.5. Servo Control Interface	46
6.6. Sensor Subsystem	47
6.7. GPIO Expansion Module	47
6.8. Power Distribution Architecture	47
6.9. Signal Integrity Considerations	47
6.10. Fail-Safe and Protection Mechanisms	48
6.11. Hardware–Firmware Consistency	48
6.12. Scalability and Expansion Potential	48
6.13. Summary	48
6.14. Performance	49
6.14.1. Purpose	49
6.15. Design decisions.	49
6.15.1. Purpose	49
6.16. Future work	49
6.16.1. Purpose	49

Capítulo 1

System Overview

1.1. Purpose of the System

The ESP32 Bluetooth RC Controller system is a modular embedded firmware platform designed to enable real-time remote control of an RC vehicle using bidirectional Bluetooth communication. The system is built around the ESP32 microcontroller and establishes a Serial Port Profile (SPP) Bluetooth link with an Android application, allowing both command transmission and telemetry feedback over a single wireless channel.

1.1.1. What the System Does

The system receives control commands from an Android application via Bluetooth SPP and translates them into hardware actions such as:

- Steering servo positioning
- Dual motor speed control
- Camera pan control
- LED activation
- Buzzer output
- Digital switch outputs

In addition to executing control commands, the firmware continuously gathers system and sensor data and transmits telemetry information back to the Android application. This telemetry may include panel values, indicators, plot data streams, input states, and sensor readings.

The communication is fully bidirectional, enabling real-time control and monitoring within the same wireless link.

1.1.2. Who Uses the System

The system is intended primarily for:

- Students learning embedded systems and wireless communication

- Makers building custom RC vehicles or robotics projects
- Hobbyists exploring ESP32-based real-time control systems
- Developers prototyping modular Bluetooth control architectures

The firmware structure is designed to be readable, extendable, and educational. Its modular organization allows users to understand how individual subsystems interact while maintaining separation of responsibilities between communication, control logic, telemetry, and hardware abstraction.

1.1.3. What Problem the System Solves

Traditional RC systems rely on dedicated RF transmitters with limited customization and minimal feedback capabilities. In contrast, this system leverages the computational power and connectivity of the ESP32 to provide:

- Real-time software-defined control logic
- Customizable packet-based communication
- Integrated telemetry feedback to a mobile device
- Expandable sensor and I²C device support

By using Bluetooth SPP as a communication layer, the system eliminates the need for proprietary transmitters and allows control from a widely available Android device.

1.1.4. Bidirectional Bluetooth Communication

The firmware establishes a Serial Port Profile (SPP) Bluetooth connection, enabling structured packet-based communication between the ESP32 and the Android application. This approach allows:

- Deterministic command reception
- Structured packet decoding
- Telemetry transmission using defined packet formats
- Synchronization and checksum validation

The use of SPP provides sufficient bandwidth for control and telemetry while maintaining implementation simplicity for educational and maker-level projects.

1.1.5. Real-Time Control and Telemetry Feedback

The system is designed around a continuous polling loop architecture that avoids blocking delays. Control packets are processed immediately upon reception, and hardware outputs are updated in deterministic order.

Simultaneously, telemetry modules collect data from sensors, input states, and internal system variables and transmit them back to the Android application. This creates a closed-loop control and monitoring environment suitable for experimentation, diagnostics, and performance tuning.

1.1.6. Modular and Expandable Architecture

A key objective of the system is modularity. Each functional domain is separated into dedicated modules, including:

- Bluetooth communication
- Packet handling and decoding
- Receiver logic
- Control processing
- Telemetry generation
- Input handling
- I²C bus and sensor management
- Debug utilities

This separation allows students and makers to modify or extend specific subsystems without affecting the entire firmware.

The architecture also supports expandability through:

- I²C-based sensors
- MCP23017 GPIO expansion
- Additional telemetry packet types
- Custom control logic modules

As a result, the system can evolve from a simple RC controller into a more advanced robotic or telemetry platform while preserving structural clarity.

1.2. System Objectives

The ESP32 Bluetooth RC Controller firmware was developed with clear engineering objectives in mind. Beyond enabling basic remote control functionality, the system is structured to serve as an educational platform for students, hobbyists, and makers who wish to understand real-time embedded communication and modular firmware design.

The following objectives guided the architecture and implementation of the system.

1.2.1. Deterministic Control Loop

The firmware is built around a continuous, non-blocking main loop executed on the ESP32. The execution order is explicitly defined, and no blocking delays are used within the control path.

This design ensures that:

- Control packets are processed in predictable order
- Hardware outputs are updated consistently
- Telemetry generation does not interfere with control execution

- The system behavior remains repeatable under normal operating conditions

For educational purposes, this deterministic structure allows students to clearly trace how data flows from communication input to physical hardware output.

1.2.2. Low-Latency Command Processing

A primary objective of the system is to minimize the delay between user input on the Android application and hardware response on the vehicle.

To achieve this:

- Bluetooth SPP is used for direct serial-style communication
- Fixed-size control packets allow fast decoding
- The receiver module processes packets immediately upon validation
- Control outputs are updated within the same execution cycle

This low-latency architecture makes the platform suitable for real-time RC control while remaining simple enough for learning and experimentation.

1.2.3. Robust Packet Synchronization

Wireless communication is inherently susceptible to noise, partial transmissions, and synchronization loss. To address this, the firmware implements structured packet framing and checksum validation.

The objectives of the packet handling design are:

- Reliable detection of packet boundaries
- Rolling buffer synchronization
- Checksum-based validation of received data
- Graceful recovery from corrupted or incomplete packets

This approach provides practical exposure to real-world communication challenges while maintaining implementation clarity for students and makers.

1.2.4. Modular Firmware Structure

The firmware is intentionally divided into independent modules, each responsible for a specific functional domain, such as:

- Bluetooth communication
- Packet definition and decoding
- Receiver logic
- Control processing
- Telemetry generation
- Input handling

- I²C communication
- Debug utilities

This modular organization enables:

- Clear separation of responsibilities
- Easier debugging and testing
- Incremental learning of subsystems
- Simplified feature expansion

For academic and DIY audiences, this structure makes the firmware easier to study, modify, and extend.

1.2.5. Hardware Abstraction

The system employs a hardware abstraction approach in which control logic is separated from direct GPIO and peripheral manipulation.

By isolating hardware-specific operations behind abstraction functions:

- Control algorithms remain independent of pin assignments
- Hardware configurations can be modified without rewriting logic
- The firmware can be adapted to different vehicle platforms

This design encourages good embedded software practices and introduces learners to scalable firmware architecture principles.

1.2.6. Expandability and Experimentation

Another central objective of the project is expandability. The firmware supports additional peripherals and features without requiring major structural modifications.

The system allows extension through:

- I²C sensor integration
- MCP23017 GPIO expansion
- Additional telemetry packet types
- Custom control logic modules
- Feature configuration through compile-time flags

This expandability makes the platform suitable not only for RC vehicles, but also for robotics experimentation, telemetry research, and embedded systems education.

By combining deterministic execution, structured communication, modular organization, and extensibility, the ESP32 Bluetooth RC Controller is engineered as both a functional control system and a practical learning platform.

1.3. High-Level System Description

The ESP32 Bluetooth RC Controller system is structured as a layered embedded architecture that separates communication, processing, and hardware interaction into clearly defined functional blocks. This separation improves clarity, maintainability, and educational value.

At a high level, the system consists of six major functional layers that operate together to provide real-time remote control and telemetry feedback.

1.3.1. Android Application

The Android application acts as the human interface of the system. It generates control commands based on user interaction (such as joystick movement, button presses, or slider adjustments) and transmits them wirelessly to the ESP32.

In addition to sending commands, the application receives telemetry data from the vehicle, allowing real-time monitoring of system variables, indicators, and sensor readings.

1.3.2. Bluetooth Communication Layer

The Bluetooth Communication Layer establishes a Serial Port Profile (SPP) connection between the Android device and the ESP32.

This layer is responsible for:

- Wireless data transmission
- Byte-level data reception
- Bidirectional serial-style communication

It provides a transparent transport channel over which structured packets are exchanged.

1.3.3. Protocol Layer

The Protocol Layer defines how raw byte streams are interpreted as structured packets. It ensures that transmitted data follows defined framing rules, includes validation mechanisms such as checksums, and maintains synchronization even in the presence of corrupted or incomplete transmissions.

This layer converts incoming byte streams into meaningful control data and formats outgoing telemetry data into structured packets.

1.3.4. Control Processing Layer

The Control Processing Layer interprets validated control packets and translates them into logical control actions.

Responsibilities include:

- Mapping joystick values to motor speeds
- Converting input data into servo positions

- Activating LEDs, buzzers, and digital outputs
- Managing control state updates

This layer contains the core decision-making logic that determines how received commands affect the vehicle's behavior.

1.3.5. Telemetry Processing Layer

The Telemetry Processing Layer gathers system information and prepares it for transmission back to the Android application.

Telemetry may include:

- Sensor measurements
- Internal system states
- Input states
- Indicator and panel values
- Plot data streams

This layer ensures that feedback information is structured and transmitted without interfering with the real-time control path.

1.3.6. Hardware Abstraction Layer

The Hardware Abstraction Layer isolates direct hardware access from higher-level control logic.

It provides standardized interfaces for:

- PWM generation for motors and servos
- Digital output control
- I²C communication
- GPIO expansion devices

By separating hardware-specific operations from logic, the system becomes easier to modify, port, and extend.

1.3.7. Physical Hardware

The Physical Hardware layer represents the actual electronic and electromechanical components connected to the ESP32. These may include:

- DC motors
- Steering and camera servos
- LEDs and buzzers
- I²C sensors
- GPIO expanders (e.g., MCP23017)

This layer interacts directly with the environment and represents the tangible output of the system.

1.3.8. System Block Diagram

Figure 1.1 illustrates the high-level architecture of the system and the bidirectional data flow between components.

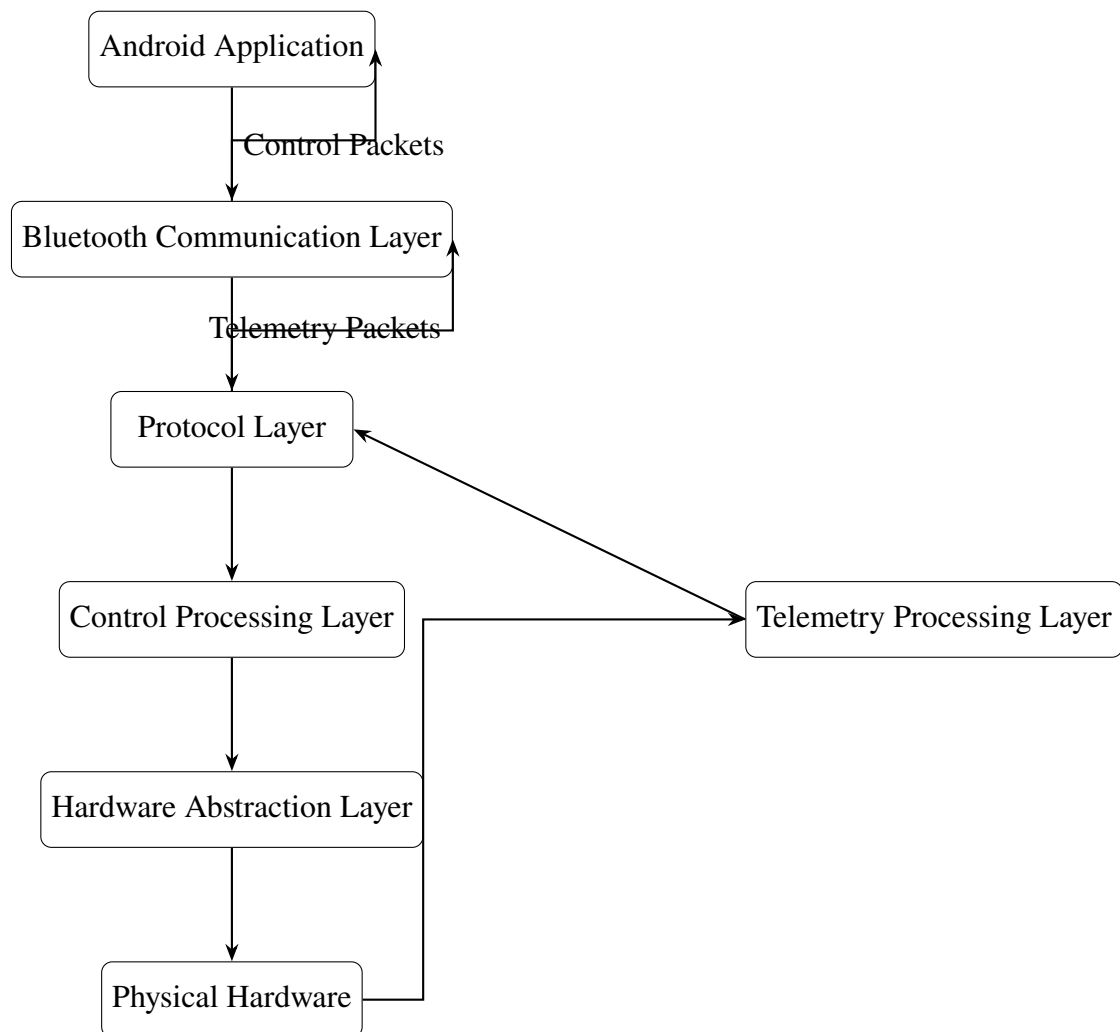


Figure 1.1: High-level system architecture and bidirectional data flow.

The diagram highlights the separation between control flow (downward path) and telemetry flow (upward path), emphasizing the bidirectional and layered nature of the system.

1.4. Functional Architecture

The system operates through two primary functional flows that execute continuously during runtime:

1. Downlink (Control Path)
2. Uplink (Telemetry Path)

These two paths form a bidirectional communication loop between the Android application and the physical hardware connected to the ESP32. The downlink path is responsible for real-time control, while the uplink path provides monitoring and feedback.

1.4.1. Downlink: Control Path

The downlink path begins at the Android application, where user interactions such as joystick movements, button presses, and slider adjustments generate control data.

The functional flow can be summarized as:

Android Application → Bluetooth → Receiver → Control → Hardware

1. Android Application

The application encodes user input into structured control packets and transmits them over the Bluetooth Serial Port Profile (SPP).

2. Bluetooth Communication

The ESP32 receives the incoming byte stream through its Bluetooth interface. The data is passed to the internal communication system for processing.

3. Receiver

The receiver component performs packet synchronization and validation. It detects packet boundaries, verifies integrity using checksum validation, and extracts meaningful control data.

Only validated packets are forwarded to the control layer.

4. Control Processing

The control layer interprets decoded packet data and translates it into logical commands. This includes mapping joystick values to PWM signals, activating digital outputs, and updating internal state variables.

5. Physical Hardware

The hardware abstraction mechanisms convert logical control commands into electrical signals that drive:

- DC motors
- Steering servos
- Camera servos
- LEDs and buzzers
- Digital outputs and expanders

This entire sequence occurs within the main execution loop, ensuring deterministic and low-latency behavior.

1.4.2. Uplink: Telemetry Path

While control commands flow from the Android device to the vehicle, telemetry data flows in the opposite direction to provide feedback and monitoring.

The functional flow can be summarized as:

Hardware → Telemetry Source → Telemetry Module → Bluetooth → Android Application

1. Physical Hardware

Sensors and system variables generate measurable data such as input states, digital readings, and computed values.

2. Telemetry Source

Telemetry sources collect relevant system data. These sources may include:

- Sensor readings via I²C
- Input state monitoring
- Internal control variables
- Status indicators

3. Telemetry Module

The telemetry module formats collected data into structured telemetry packets. Different packet types may represent panels, indicators, plots, configuration values, or input states.

These packets are constructed according to the defined protocol format and prepared for transmission.

4. Bluetooth Communication

Formatted telemetry packets are transmitted over the same Bluetooth SPP connection used for control commands.

5. Android Application

The Android application receives and decodes telemetry packets, updating graphical elements such as indicators, charts, and data panels in real time.

1.4.3. Bidirectional Operation

The downlink and uplink paths operate concurrently within the same system loop. Control commands and telemetry packets share the Bluetooth communication channel but are structured to avoid ambiguity.

This bidirectional architecture enables:

- Real-time remote control
- Continuous system monitoring
- Closed-loop experimentation
- Interactive debugging and tuning

By clearly separating control processing from telemetry generation while allowing both to operate simultaneously, the system achieves responsive behavior without sacrificing modularity or clarity.

This functional dual-path structure is fundamental to the overall architecture of the ESP32 Bluetooth RC Controller.

1.5. Real-Time Characteristics

The ESP32 Bluetooth RC Controller firmware is designed as a real-time embedded control system operating without a real-time operating system (RTOS). Instead, it relies on a structured polling-based architecture and a deterministic execution loop to ensure predictable and responsive behavior.

This section describes the real-time design principles that guide the firmware implementation.

1.5.1. Polling-Based Architecture

The system operates using a continuous main loop that repeatedly executes core functional tasks in a defined order. Rather than relying on complex task scheduling or interrupt-driven processing for high-level logic, the firmware uses structured polling to evaluate communication, control updates, and telemetry generation.

Within each loop iteration, the system:

- Checks for incoming Bluetooth data
- Processes and validates complete packets
- Updates control outputs
- Generates telemetry data when required

This polling-based approach simplifies system reasoning and makes execution flow easier to understand for students and hobbyists learning embedded systems design.

1.5.2. Non-Blocking Loop Design

A key objective of the firmware is to maintain continuous system responsiveness. For this reason, the main loop avoids blocking operations that could delay control updates or communication processing.

The design ensures that:

- Communication handling does not stall control updates
- Telemetry transmission does not interrupt command processing
- No long-duration waiting operations are introduced in the control path

By avoiding blocking constructs, the system maintains consistent loop execution frequency and responsive behavior.

1.5.3. Avoidance of `delay()` Usage

The firmware intentionally avoids the use of `delay()` in the primary execution path. Blocking delays introduce non-deterministic timing behavior and can significantly degrade responsiveness in real-time control applications.

Instead, time-dependent behavior is handled through:

- Periodic checks within the loop

- Conditional execution logic
- Lightweight timing control mechanisms

This approach ensures that the system remains reactive to incoming control packets at all times.

1.5.4. Deterministic Execution Order

The execution order of functional blocks inside the loop is explicitly defined and consistent across iterations. Each iteration processes communication, control logic, and telemetry in the same sequence.

This deterministic structure provides:

- Predictable system behavior
- Easier debugging and tracing
- Stable control response characteristics
- Clear educational value for understanding data flow

Because the order of operations does not change dynamically, system timing can be analyzed and estimated more easily, which is essential for later performance evaluation.

1.5.5. Fixed-Size Packet Model for Control

Control packets are designed with a fixed-size structure. This simplifies parsing, reduces conditional branching during decoding, and ensures that control updates require a predictable amount of processing time.

The fixed-size packet model provides:

- Constant-time decoding complexity
- Reduced memory fragmentation
- Faster synchronization within the receiver
- Deterministic control update latency

By combining fixed-size control packets with structured validation and deterministic loop execution, the system achieves consistent real-time performance suitable for RC vehicle applications.

1.5.6. Preparation for Timing Analysis

The architectural decisions described in this section establish a foundation for later timing and performance analysis. Because the firmware follows a polling-based, non-blocking, and deterministic structure, it becomes possible to estimate:

- Control response latency
- Maximum packet processing rate
- Telemetry throughput limits
- Loop execution frequency

These characteristics are examined in more detail in subsequent chapters.

1.6. Modular Firmware Philosophy

A central design principle of the ESP32 Bluetooth RC Controller is modularity. The firmware is intentionally organized into independent functional modules, each encapsulated within its own header (.h) and implementation (.cpp) files.

This modular structure promotes clarity, maintainability, and scalability, while also serving as an educational example of structured embedded firmware development.

1.6.1. Separation of Responsibilities

Each subsystem of the firmware is responsible for a clearly defined functional domain. This separation prevents cross-dependencies between unrelated components and simplifies debugging and feature expansion.

The major functional domains include:

- **Bluetooth Communication** — Handles wireless data transmission and reception using the Serial Port Profile (SPP).
- **Receiver** — Processes incoming byte streams, performs packet synchronization, validates checksums, and extracts structured control data.
- **Packet Definitions** — Defines structured data formats for control and telemetry communication, ensuring consistent interpretation between the ESP32 and the Android application.
- **Control Processing** — Translates validated control data into logical commands that affect motors, servos, and digital outputs.
- **Input Handling** — Manages physical or logical input states and integrates them into telemetry or control logic.
- **Telemetry System** — Collects system data, formats telemetry packets, and transmits structured feedback to the Android application.
- **I²C Communication** — Manages external sensors and peripheral devices connected via the I²C bus.
- **Debug Utilities** — Provides optional diagnostic output and compile-time debugging tools without interfering with core functionality.

This explicit separation ensures that changes within one module do not unintentionally affect unrelated parts of the system.

1.6.2. Hardware Abstraction Separation

In addition to functional modularity, the firmware applies hardware abstraction principles. Direct manipulation of GPIO pins, PWM outputs, and communication peripherals is isolated from higher-level control logic.

By separating hardware-specific implementation details from system logic:

- Control algorithms remain independent of physical pin assignments
- The firmware can be adapted to different hardware configurations
- Platform-specific changes are localized within defined layers

This abstraction encourages best practices in embedded systems design and makes the project suitable as a teaching example.

1.6.3. Educational and Practical Benefits

For students and makers, a modular architecture offers several advantages:

- Easier understanding of data flow between subsystems
- Simplified experimentation with individual modules
- Reduced risk when modifying or extending functionality
- Clear mapping between conceptual layers and implementation files

Instead of a monolithic firmware structure, the system demonstrates how larger embedded projects can be organized into manageable components.

1.6.4. Scalability and Future Extension

The modular philosophy allows new capabilities to be added without restructuring the entire firmware. For example:

- Additional telemetry packet types can be introduced independently
- New sensors can be integrated through the I²C subsystem
- Alternative communication layers could replace Bluetooth without rewriting control logic
- Hardware outputs can be expanded using GPIO expanders

This scalability ensures that the firmware can evolve from a simple RC controller into a more advanced experimental or educational platform.

By combining clear responsibility separation with hardware abstraction and extensibility, the modular firmware philosophy forms a foundational element of the ESP32 Bluetooth RC Controller architecture.

1.7. System Capabilities

The ESP32 Bluetooth RC Controller firmware provides a wide range of functional capabilities intended for real-time vehicle control, monitoring, and experimentation. These capabilities support both practical RC applications and educational exploration of embedded system design.

The system is designed to be flexible and extensible while maintaining clarity and structured behavior.

1.7.1. Motion Control

The firmware supports real-time motion control of an RC platform, including:

- **Dual Motor Control** — Independent control of two DC motors for differential drive or tracked vehicle configurations.
- **Steering Servo Control** — Precise positioning of a steering servo for directional control.
- **Camera Pan Servo Control** — Independent servo control for camera positioning or auxiliary mechanical movement.

These motion control capabilities enable a wide range of vehicle configurations, from simple RC cars to more advanced robotic platforms.

1.7.2. Digital Output Control

The system provides support for auxiliary digital outputs, including:

- **LED Indicators** — Control of visual indicators for status signaling or decorative purposes.
- **Buzzer Control** — Activation of an audible buzzer for alerts or feedback.
- **Switch Outputs** — General-purpose digital outputs that can be used to control relays, lights, or additional actuators.

These outputs extend the system beyond basic motion control, allowing richer interaction and experimentation.

1.7.3. Sensor Integration

The firmware supports external sensor integration to enhance system awareness and telemetry feedback.

- **I²C Sensor Support** — Integration of digital sensors connected through the I²C bus.
- **MCP23017 GPIO Expansion** — Expansion of available digital input/output channels using an external GPIO expander device.

These capabilities enable users to monitor environmental data, extend digital inputs/outputs, and experiment with additional hardware modules.

1.7.4. Telemetry and Monitoring Features

The system provides structured telemetry functionality to enable real-time monitoring from the Android application. Supported telemetry features include:

- **Configurable Telemetry Panels** — Display of structured numerical or textual data values.
- **Plot Telemetry Streams** — Continuous data streaming for graphical plotting and analysis.
- **Indicator Telemetry** — Real-time status indicators for binary or state-based information.
- **Input Telemetry Reporting** — Feedback of input states and system variables to the Android interface.

These telemetry features create a closed-loop control and monitoring environment, allowing users to observe system behavior while actively controlling the vehicle.

1.7.5. Platform Flexibility

By combining motion control, digital outputs, sensor integration, and structured telemetry, the firmware supports:

- Differential drive vehicles
- Servo-steered vehicles
- Telemetry-enabled robotics experiments
- Educational embedded systems projects
- DIY hobbyist RC builds

The system capabilities are intentionally broad to encourage experimentation, learning, and incremental project expansion.

1.8. System Constraints

While the ESP32 Bluetooth RC Controller firmware provides flexible control and telemetry capabilities, it operates within practical hardware and communication constraints. Understanding these limitations is essential for proper system design, performance evaluation, and future expansion.

1.8.1. Bluetooth Bandwidth and Latency

The system relies on Bluetooth Serial Port Profile (SPP) for bidirectional communication. Although sufficient for real-time RC control and structured telemetry, Bluetooth SPP introduces inherent constraints:

- Limited bandwidth compared to wired communication
- Variable latency depending on signal quality and distance
- Single active connection model in typical usage

These factors influence maximum telemetry throughput and practical control update rates.

1.8.2. Microcontroller Resource Limits

The ESP32 microcontroller provides substantial computational capability for hobby and educational applications; however, it remains a resource-constrained embedded device.

System limitations include:

- Finite RAM for packet buffers and telemetry data
- Limited stack and heap space
- Shared CPU time between communication, control, and telemetry tasks

Careful memory usage and fixed-size packet structures are used to ensure predictable operation within these limits.

1.8.3. Fixed Buffer Sizes

Packet reception and processing rely on predefined buffer sizes. While this simplifies memory management and maintains deterministic behavior, it introduces constraints:

- Maximum packet size is bounded
- Telemetry payload expansion must remain within defined limits
- Buffer overflow risks must be considered during future modifications

These design choices favor reliability and simplicity over dynamic allocation flexibility.

1.8.4. Checksum Simplicity

The communication protocol uses a lightweight checksum mechanism for packet validation. While suitable for educational and hobby-level applications, this approach does not provide the same robustness as more advanced error detection techniques such as CRC.

As a result:

- The system tolerates minor transmission errors
- It may not detect all possible data corruption patterns
- It is optimized for simplicity rather than industrial-grade reliability

This trade-off aligns with the project's educational and experimental objectives.

1.8.5. Single-Loop Execution Model

The firmware operates within a single main execution loop without a full real-time operating system managing independent tasks.

Consequences of this design include:

- All processing shares the same execution context
- Excessive telemetry or additional features may reduce loop frequency
- Care must be taken when adding time-intensive operations

However, this simplified model enhances clarity and makes the firmware easier to study and modify.

1.8.6. Hardware Dependency

Although the firmware uses hardware abstraction principles, it is designed specifically for the ESP32 platform. Porting the system to another microcontroller would require:

- Adaptation of Bluetooth communication mechanisms
- Reconfiguration of PWM and GPIO control

- Adjustment of timing assumptions

Therefore, while modular, the system remains tied to the ESP32 hardware ecosystem.

1.8.7. Educational Scope

The firmware is intentionally designed for academic, recreational, and DIY applications. It is not intended for:

- Safety-critical control systems
- Industrial automation
- High-reliability commercial deployments

Its primary objective is to provide a structured, understandable, and expandable embedded communication platform for learning and experimentation.

Recognizing these constraints ensures responsible use of the system and establishes realistic expectations for performance and reliability.

1.9. Document Structure

This document is organized to progressively guide the reader from high-level system concepts to detailed architectural and implementation aspects. The structure follows a top-down approach, beginning with system-level descriptions and advancing toward protocol, firmware architecture, and performance considerations.

The chapters are organized as follows:

- **Chapter 1 – System Overview**
Introduces the purpose, objectives, architecture, capabilities, constraints, and real-time characteristics of the system. This chapter establishes the conceptual foundation for understanding the platform.
- **Chapter 2 – System Architecture**
Describes the layered architectural model in greater detail, including the interaction between communication, processing, and hardware abstraction layers.
- **Chapter 3 – Communication Protocol Specification**
Defines the packet structure, framing rules, synchronization strategy, checksum mechanism, and telemetry formats used for bidirectional communication.
- **Chapter 4 – Firmware Architecture**
Maps the conceptual architecture to the actual firmware organization, describing the responsibilities and interactions of each module.
- **Chapter 5 – Real-Time Behavior and Timing Analysis**
Examines loop execution characteristics, control latency, telemetry throughput, and performance considerations derived from the deterministic execution model.
- **Chapter 6 – Hardware Integration and Mapping**
Describes the relationship between firmware outputs and physical hardware components, including motors, servos, sensors, and expansion devices.

- **Chapter 7 – Memory and Performance Considerations**

Discusses resource utilization, buffer sizing strategies, packet efficiency, and trade-offs between simplicity and robustness.

- **Chapter 8 – Design Decisions and Rationale**

Explains the engineering choices made during development, including communication strategy, modular design approach, and architectural trade-offs.

- **Chapter 9 – Future Work and Expansion Opportunities**

Outlines potential improvements, scalability paths, and research directions for extending the system.

This structured progression ensures that readers with different backgrounds—academic, hobbyist, or engineering-focused—can follow the material from conceptual understanding to technical depth in a logical and systematic manner.

Capítulo 2

Firmware Architecture

2.1. Architectural Objectives

The firmware architecture is designed to enforce structural clarity, execution determinism, and controlled modular growth.

The architectural objectives are:

- Enforce separation between communication, control logic, and hardware interaction.
- Maintain deterministic execution ordering.
- Constrain inter-module dependencies.
- Enable incremental feature expansion without structural degradation.
- Isolate hardware-specific implementation details.

These objectives define the structural organization of the firmware.

2.2. Architectural Decomposition Model

The firmware follows a layered decomposition strategy. Each layer encapsulates a specific abstraction level and interacts only with adjacent layers.

2.2.1. Logical Layers

The system is partitioned into four logical layers:

1. Application / Control Layer
2. Communication Layer
3. Hardware Abstraction Layer (HAL)
4. Physical Interface Layer

Application / Control Layer

This layer contains system-level logic and internal state management. It does not access hardware registers or communication buffers directly.

Communication Layer

This layer manages structured packet exchange and data validation. It translates external serialized data into internal control variables and formats telemetry structures for transmission.

Hardware Abstraction Layer

The HAL provides a stable interface between logical control variables and microcontroller peripherals. All direct peripheral configuration and register-level interaction is confined to this layer.

Physical Interface Layer

Represents electrical connections and external components. No firmware logic exists at this layer.

2.3. Module-Level Decomposition

Within the layered structure, the firmware is divided into discrete modules. Each module has a well-defined responsibility and limited external visibility.

Primary module categories include:

- Communication Parsing Module
- Control Variable Management Module
- Motor Control Module
- Servo Control Module
- Telemetry Aggregation Module
- Sensor Interface Modules
- GPIO Expansion Driver
- Output Management Module
- Debug and Diagnostic Module

Modules interact through controlled data interfaces rather than direct cross-calls.

2.4. Execution Model

The firmware executes within a single main control loop. No preemptive scheduler or real-time operating system is employed.

2.4.1. Deterministic Ordering

Each loop iteration executes subsystems in a fixed order. The ordering is intentionally static to guarantee predictable behavior and bounded execution time.

2.4.2. Polling Strategy

Subsystems are serviced through structured polling. This eliminates concurrency conflicts and reduces synchronization complexity.

2.4.3. Non-Blocking Constraint

Blocking delays are avoided within loop-executed modules. All time-dependent behavior must be implemented using state tracking rather than delay-based waiting.

This constraint ensures bounded worst-case loop duration.

2.5. Data Ownership and State Management

The firmware maintains a centralized internal state model.

2.5.1. State Variables

State variables represent:

- Target actuator values
- Measured sensor data
- Communication status
- Output states

Each module is permitted to modify only the state variables within its defined responsibility domain.

2.5.2. State Update Discipline

State updates occur in deterministic phases of the control loop. Read-modify-write patterns are controlled to avoid unintended side effects.

2.6. Data Flow Structure

2.6.1. Command Processing Path

Incoming serialized data is:

1. Received into a communication buffer
2. Validated and parsed
3. Translated into internal control variables

4. Applied to actuator update logic

2.6.2. Telemetry Processing Path

Telemetry data is:

1. Collected from state variables and sensor modules
2. Assembled into structured packets
3. Serialized into a transmission buffer
4. Sent through the communication interface

Buffers isolate communication timing from control execution timing.

2.7. Interface Boundaries

Strict interface boundaries are enforced between:

- Communication and Control logic
- Control logic and HAL
- HAL and hardware peripherals

Direct cross-layer access is prohibited. This containment prevents structural coupling and simplifies future refactoring.

2.8. Hardware Abstraction Strategy

All hardware-specific configuration is localized within the HAL.

Abstraction mechanisms include:

- Encapsulated peripheral initialization
- Driver-level isolation
- Pin mapping centralization
- Replaceable hardware drivers

Control modules remain independent of specific peripheral registers or microcontroller configuration details.

2.9. Scalability and Controlled Expansion

The architecture supports feature growth through modular extension.

2.9.1. Sensor Integration

New sensors are introduced via independent interface modules connected through the HAL.

2.9.2. Actuator Integration

New actuators are implemented as discrete control modules without altering communication parsing logic.

2.9.3. I/O Expansion

External expansion devices increase hardware capacity while maintaining logical separation from application logic.

2.9.4. Structural Stability

Architectural integrity is preserved by:

- Maintaining layer boundaries
- Avoiding circular dependencies
- Restricting module visibility

These constraints prevent architectural degradation as the firmware evolves.

Capítulo 3

Communication Protocol Specification

3.1. Protocol Overview

This chapter defines the structured packet-based communication protocol used for bidirectional data exchange between the embedded firmware and the external Android application.

The protocol is designed to achieve:

- Deterministic packet parsing
- Robust synchronization recovery
- Low processing overhead
- Fixed-length command handling
- Structured telemetry formatting

All communication occurs over a Bluetooth serial transport layer. The protocol operates independently of the underlying physical transport once a byte stream is established.

3.2. Communication Model

The communication model is packet-oriented. All data exchanged between the application and firmware is encapsulated within structured packets.

Two packet categories exist:

- Control Packets (Downlink)
- Telemetry Packets (Uplink)

The firmware parser continuously monitors the incoming byte stream and attempts to detect valid packet frames.

3.3. Packet Framing Strategy

To guarantee reliable synchronization within a continuous byte stream, each packet contains:

- A Start-of-Frame (SOF) marker
- A Payload section
- A Checksum field

3.3.1. General Packet Structure

SOF	Payload	Checksum
-----	---------	----------

3.3.2. Start-of-Frame Marker

The SOF marker is a predefined byte sequence used to identify the beginning of a valid packet.

Its purpose is to:

- Allow parser resynchronization after byte loss
- Prevent payload misinterpretation
- Reduce false-positive packet detection

The SOF pattern is unique and not permitted to appear as a valid packet prefix.

3.4. Control Packet Format

Control packets are transmitted from the Android application to the firmware.

3.4.1. Control Packet Structure

SOF	Control Fields	Reserved	Checksum
-----	----------------	----------	----------

3.4.2. Control Fields

The control payload contains fixed-size fields representing actuator commands and output states.

Typical control fields may include:

- Motor speed values
- Steering position
- Auxiliary servo position
- Output enable flags

Each field has:

- Defined byte width
- Defined numerical range
- Defined scaling factor

All control packets are fixed-length to simplify parsing logic and maintain constant processing time.

3.5. Telemetry Packet Format

Telemetry packets are transmitted from the firmware to the Android application.

3.5.1. Telemetry Packet Structure

SOF	Telemetry Fields	Status Flags	Checksum
-----	------------------	--------------	----------

3.5.2. Telemetry Fields

Telemetry payload may include:

- Sensor measurements
- Actuator state feedback
- Input states
- System indicators

Telemetry structure is deterministic and ordered to ensure consistent parsing on the application side.

3.6. Checksum and Error Detection

Each packet terminates with a checksum byte.

3.6.1. Checksum Purpose

The checksum enables:

- Detection of corrupted packets
- Rejection of malformed data
- Prevention of unsafe actuator updates

3.6.2. Checksum Model

The checksum is computed as a deterministic function of all payload bytes:

$$Checksum = f(b_1, b_2, \dots, b_n)$$

Where:

- b_i represents payload bytes
- f is a defined aggregation function (e.g., modulo sum)

Packets failing checksum validation are discarded without affecting actuator states.

3.7. Parser Synchronization Model

The parser operates as a state machine with the following states:

1. Idle (waiting for SOF)
2. Receiving Payload
3. Validating Checksum
4. Packet Accepted / Rejected

If checksum validation fails, the parser returns to the Idle state and resumes searching for the next SOF marker.

This ensures recovery from:

- Byte loss
- Transmission noise
- Partial packet corruption

3.8. Timing Characteristics

Control packets are processed within a single control loop iteration once fully received and validated.

Telemetry packets are generated at a deterministic rate defined by the control loop frequency.

The protocol avoids variable-length parsing to ensure bounded processing time.

3.9. Protocol Determinism

The protocol enforces determinism through:

- Fixed packet length
- Defined byte ordering
- Consistent field offsets
- Stateless packet interpretation

This guarantees constant-time decoding complexity.

3.10. Extensibility Strategy

Protocol extensibility is achieved through:

- Reserved payload bytes
- Version identifier field (optional)
- Backward-compatible field extension

Future fields may be appended without altering parser stability if packet size constraints are maintained.

3.11. Protocol Robustness Considerations

The protocol design accounts for:

- Bluetooth latency variability
- Occasional packet loss
- Transmission jitter

No partial command execution occurs. Only fully validated packets update control variables.

This prevents unintended actuator behavior.

3.12. Summary

The communication protocol provides:

- Structured bidirectional packet exchange
- Deterministic decoding behavior
- Error detection via checksum validation
- Synchronization recovery capability
- Fixed processing complexity

The following chapter details how this protocol is implemented within the firmware module structure.

Capítulo 4

Firmware Structure

4.1. Introduction

This chapter describes the concrete organization of the firmware source code and the structural implementation of the architectural model defined in Chapter 2.

While Chapter 2 defines logical layers and architectural principles, this chapter explains how those principles are realized in the actual firmware structure.

The firmware is organized using a modular C++ design, with each functional subsystem encapsulated in dedicated header and source files.

4.2. Source Code Organization Strategy

The firmware follows a structured separation between:

- Interface definitions (.h files)
- Implementation logic (.cpp files)

Each module exposes only its public interface through its header file, while internal logic remains confined to its implementation file.

This separation enforces:

- Controlled visibility
- Reduced coupling
- Clear responsibility boundaries

4.3. Main Entry Point Structure

The firmware entry point consists of the standard embedded structure:

- System initialization phase
- Continuous execution loop

4.3.1. Initialization Phase

During initialization:

- Communication interfaces are configured
- Peripheral drivers are initialized
- GPIO and expansion modules are configured
- Internal state variables are set to default values

All hardware setup is completed before entering the main execution loop.

4.3.2. Main Control Loop

The main loop orchestrates subsystem execution in deterministic order.

Conceptually, the loop performs:

1. Communication processing
2. Control state updates
3. Actuator refresh
4. Sensor acquisition
5. Telemetry generation

No blocking delays are permitted inside loop-executed modules.

4.4. Module Categories

The firmware is decomposed into the following major module categories.

4.4.1. Communication Module

Responsible for:

- Receiving raw byte streams
- Detecting packet framing
- Validating checksums
- Extracting structured command data
- Serializing telemetry packets

The communication module does not directly modify hardware outputs. It only updates internal control variables after validation.

4.4.2. Control and Input Module

This module translates validated command data into internal state variables.

Responsibilities include:

- Updating target motor speeds
- Updating servo positions
- Updating output flags

It does not perform direct peripheral access.

4.4.3. Motor Control Module

Encapsulates motor driver interactions.

Responsibilities:

- PWM signal generation
- Direction control
- Safe state fallback handling

All motor-related hardware configuration is isolated here.

4.4.4. Servo Control Module

Handles generation of position signals for:

- Steering servo
- Auxiliary servo

Servo timing precision is maintained independently from communication timing.

4.4.5. Telemetry Module

Aggregates data from:

- Sensor interfaces
- Actuator state variables
- Input state variables
- System indicators

This module prepares structured telemetry packets for transmission.

4.4.6. Sensor Interface Modules

Each sensor is encapsulated within its own driver module.

Responsibilities:

- Sensor initialization
- Data acquisition
- Data scaling and formatting

Sensor modules expose processed measurements to the telemetry module.

4.4.7. GPIO Expansion Driver

Handles communication with external GPIO expansion hardware.

Responsibilities:

- I2C communication management
- Remote digital input reading
- Remote digital output control

Expansion logic is isolated from application control logic.

4.4.8. Output Management Module

Manages auxiliary outputs such as:

- LEDs
- Buzzer
- Switch outputs

Outputs are updated deterministically during each control loop iteration.

4.5. Inter-Module Communication Model

Modules communicate through:

- Shared state variables
- Controlled function interfaces

Direct cross-module register access is prohibited.

This ensures:

- Predictable data ownership
- Reduced unintended side effects
- Simplified debugging

4.6. Header File Discipline

Header files:

- Contain only declarations

- Avoid implementation logic
- Expose minimal necessary interfaces

This reduces compilation dependencies and enforces modular encapsulation.

4.7. Configuration and Pin Isolation

Pin definitions and hardware configuration constants are centralized in dedicated configuration files.

This ensures:

- Hardware portability
- Simplified hardware replacement
- Avoidance of hard-coded values

No module contains scattered pin assignments.

4.8. Error Handling Strategy

The firmware implements lightweight error handling mechanisms:

- Packet rejection on checksum failure
- Safe actuator state retention on communication loss
- Controlled output states during initialization

Critical error paths avoid system crashes or undefined behavior.

4.9. Resource Management Considerations

The implementation minimizes:

- Dynamic memory allocation
- Large stack usage
- Blocking delays

Buffers are statically allocated with fixed sizes to guarantee bounded memory usage.

4.10. Implementation Consistency with Architecture

The firmware implementation directly reflects the layered architecture defined in Chapter 2:

- Communication logic isolated from hardware
- Control logic isolated from peripheral registers
- Hardware abstraction centralized

- Modular growth supported

This alignment ensures architectural integrity is preserved in the concrete implementation.

4.11. Summary

The firmware structure enforces:

- Modular separation
- Deterministic execution
- Hardware abstraction
- Controlled inter-module interaction
- Expandable feature integration

The following chapter analyzes timing behavior and real-time characteristics of this implementation.

Capítulo 5

Timing and Real-Time Analysis

5.1. Introduction

This chapter analyzes the temporal behavior of the firmware execution model. Although the system does not implement a real-time operating system (RTOS), its architecture follows deterministic scheduling principles suitable for embedded control applications.

The objective of this chapter is to:

- Characterize the main control loop frequency
- Estimate worst-case execution time (WCET)
- Analyze communication latency
- Evaluate actuator update timing
- Validate system responsiveness

5.2. Execution Model Overview

The firmware operates under a single-threaded cooperative execution model:

- All modules execute sequentially
- No blocking delays are used
- No dynamic task switching occurs

The control loop repeats continuously with execution time determined by the cumulative duration of all module operations.

5.3. Control Loop Structure

The logical execution order inside the main loop is:

1. Communication processing
2. Command decoding

3. Control state update
4. Motor update
5. Servo update
6. Sensor acquisition
7. Telemetry packet generation

This order ensures that new control inputs affect actuators within the same iteration cycle.

5.4. Worst-Case Execution Time (WCET)

Worst-case execution time is defined as the maximum duration required to complete one full iteration of the main control loop.

Let:

$$T_{loop} = T_{comm} + T_{control} + T_{motor} + T_{servo} + T_{sensor} + T_{telemetry}$$

Each term represents the maximum processing time of the corresponding subsystem.

5.4.1. Qualitative Estimation

In typical operation:

- Communication parsing is lightweight (byte buffer inspection)
- Motor and servo updates involve simple register writes
- Sensor acquisition depends on I2C latency
- Telemetry formatting involves structured serialization

The dominant time contributor is typically I2C communication when accessing external peripherals.

5.5. Loop Frequency Estimation

The control loop frequency is approximated as:

$$f_{loop} = \frac{1}{T_{loop}}$$

For responsive RC control systems, a target loop frequency above 50 Hz is sufficient. Higher frequencies (100–500 Hz) improve control smoothness and reduce perceptible latency.

Given the lightweight nature of operations and the processing capability of the microcontroller, the system is expected to operate significantly above the minimum 50 Hz requirement.

5.6. Communication Latency

End-to-end command latency consists of:

- Transmission delay over Bluetooth
- Packet parsing delay
- Control loop synchronization delay

Total latency can be approximated as:

$$T_{latency} = T_{bt} + T_{parse} + T_{loop_sync}$$

Where:

- T_{bt} is wireless transmission delay
- T_{parse} is packet validation time
- T_{loop_sync} is the maximum time until next loop iteration

Because the loop runs continuously without blocking delays, T_{loop_sync} is bounded by T_{loop} .

This guarantees predictable maximum response delay.

5.7. Actuator Update Timing

Motor and servo updates occur once per loop iteration.

Therefore:

$$T_{actuator_update} \leq T_{loop}$$

This ensures:

- Deterministic actuator refresh
- No irregular pulse timing
- Stable PWM output generation

Servo control timing is hardware-assisted by internal PWM peripherals, reducing dependency on software timing precision.

5.8. Sensor Sampling Rate

Sensor sampling frequency is also determined by loop frequency:

$$f_{sensor} = f_{loop}$$

However, for sensors requiring longer acquisition time (e.g., via I2C), effective sampling frequency may be slightly lower.

Sensor reads are designed to:

- Avoid blocking delays
- Complete within a bounded execution time

5.9. Determinism Analysis

The firmware achieves determinism through:

- Fixed execution order
- Absence of random delays
- No dynamic memory allocation
- Bounded buffer sizes

This ensures repeatable timing behavior under identical operating conditions.

5.10. Jitter Considerations

Jitter refers to variations in loop execution time.

Primary jitter sources include:

- Variable Bluetooth packet arrival timing
- I2C bus contention
- Interrupt-driven background processes

However, because:

- Actuator signals use hardware PWM
- No blocking communication is used

Control stability is not significantly affected by small loop timing variations.

5.11. Real-Time Classification

The system can be classified as:

- Soft real-time

Reasoning:

- Occasional minor delays do not cause catastrophic failure
- Deadlines are important for performance but not safety-critical
- System maintains stable state during communication loss

This classification is appropriate for RC control systems.

5.12. Communication Loss Handling

If communication is interrupted:

- Last valid control state is retained
- System remains stable
- Optional timeout-based fail-safe may be implemented

Timeout detection can be expressed as:

$$T_{now} - T_{last_packet} > T_{timeout}$$

If true, actuators can be set to predefined safe states.

5.13. Scalability Impact on Timing

Adding new modules increases:

$$T_{loop}$$

However, because of modular structure:

- New features can be evaluated individually
- Timing impact can be measured incrementally
- System limits remain predictable

This supports controlled expansion without architectural degradation.

5.14. Summary

The firmware timing model provides:

- Deterministic execution order
- Bounded worst-case loop duration
- Predictable actuator update timing
- Controlled communication latency
- Soft real-time operational classification

The next chapter presents validation testing and performance measurements to empirically verify these timing characteristics.

Capítulo 6

Hardware Architecture and Implementation

6.1. Introduction

This chapter describes the physical hardware platform used to implement the firmware architecture defined in previous chapters.

The objective is to:

- Describe the electronic components used
- Justify hardware selection decisions
- Explain system-level interconnections
- Demonstrate consistency with the hardware abstraction strategy

The hardware platform was designed to support modularity, expandability, and reliable real-time operation.

6.2. System Overview

The hardware system consists of:

- A primary microcontroller unit (MCU)
- Wireless communication interface
- Motor driver stage
- Servo control outputs
- Sensor interfaces
- GPIO expansion module
- Power management circuitry

All subsystems are electrically integrated while maintaining logical separation consistent with the firmware structure.

6.3. Microcontroller Unit

The core processing element is the **ESP32**

The ESP32 was selected due to:

- Integrated Bluetooth capability
- Dual-core processing architecture
- Hardware PWM peripherals
- I2C and UART support
- Sufficient RAM and Flash resources

Its integrated wireless stack eliminates the need for external communication modules, reducing system complexity.

6.4. Motor Driver Stage

DC motor actuation is achieved through an H-bridge driver module.

The driver stage provides:

- Bidirectional current flow
- PWM speed control
- Electrical isolation between logic and motor supply

The driver was selected based on:

- Current handling capacity
- Voltage compatibility
- Switching efficiency

The logic-level control inputs are directly interfaced with MCU PWM outputs.

6.5. Servo Control Interface

Steering and auxiliary actuation use standard hobby servo motors.

Servo signals are generated via:

- Dedicated hardware PWM channels
- 50 Hz refresh frequency
- Pulse width range typically between 1 ms and 2 ms

Because servo timing is hardware-assisted, software timing jitter does not affect signal stability.

6.6. Sensor Subsystem

Sensors integrated into the system include:

- Analog voltage measurement inputs
- Digital state inputs
- I2C-based sensor devices

Analog signals are read through the MCU ADC channels, while digital expansion is handled via an external GPIO expander.

This hybrid approach increases scalability without exhausting native MCU pins.

6.7. GPIO Expansion Module

An external I2C-based GPIO expander is used to extend digital input/output capacity.

Advantages:

- Reduced pin usage on MCU
- Modular hardware scaling
- Electrical separation of subsystems

The I2C bus operates at a standard clock frequency compatible with reliable real-time operation.

6.8. Power Distribution Architecture

The system includes separate supply domains:

- Logic supply (3.3V)
- Servo supply (typically 5V)
- Motor supply (dependent on motor rating)

Separation of power domains prevents:

- Voltage dips during motor startup
- Electrical noise propagation
- Microcontroller instability

Decoupling capacitors are placed near high-current devices to reduce transient disturbances.

6.9. Signal Integrity Considerations

The following measures were taken to preserve signal integrity:

- Short PWM trace lengths
- Common ground reference

- Shielded or twisted wiring for motor leads
- Pull-up resistors for I2C lines

These precautions reduce electromagnetic interference (EMI) and improve system reliability.

6.10. Fail-Safe and Protection Mechanisms

Hardware-level protection includes:

- Reverse polarity protection (if implemented)
- Current limitations via driver module
- Brown-out detection (MCU internal feature)

These mechanisms enhance operational safety and prevent hardware damage.

6.11. Hardware–Firmware Consistency

The hardware implementation directly reflects the abstraction strategy defined in Chapter 2:

- All pins are centrally defined
- Peripheral interfaces match modular firmware drivers
- No hard-coded electrical assumptions exist in application logic

This ensures that hardware changes (e.g., driver replacement) require minimal firmware modification.

6.12. Scalability and Expansion Potential

The hardware platform supports future extensions such as:

- Additional sensors
- Higher current motor stages
- External communication interfaces
- Advanced feedback control modules

Expansion is supported through both available MCU peripherals and modular firmware structure.

6.13. Summary

The hardware architecture provides:

- Integrated wireless capability
- Deterministic actuator control

- Modular expansion support
- Electrical stability and protection
- Direct alignment with firmware abstraction principles

This cohesive hardware–software integration demonstrates a complete embedded system implementation rather than isolated firmware development.

Capítulo 7

Performance Evaluation

7.1. Introduction

This chapter presents the experimental performance evaluation of the implemented system. While Chapter 5 provided theoretical timing analysis, this chapter validates system behavior through empirical observation and measurement.

The evaluation focuses on:

- Control loop frequency
- Communication latency
- Actuator responsiveness
- Wireless stability
- Resource utilization

7.2. Experimental Setup

The system was tested under standard operating conditions:

- Bluetooth connection established with Android control application
- Motors connected under nominal load
- Servo connected for steering control
- Telemetry transmission enabled

Measurements were performed using:

- Serial debug timestamps
- Oscilloscope verification for PWM signals (if available)
- Software-based timing counters

7.3. Control Loop Frequency Measurement

The control loop frequency was measured by recording timestamps at the beginning of each iteration.

The average loop period is defined as:

$$T_{avg} = \frac{\sum_{i=1}^N (t_i - t_{i-1})}{N}$$

The loop frequency is then:

$$f_{loop} = \frac{1}{T_{avg}}$$

7.3.1. Observed Results

Under normal operation:

- Loop frequency remained stable above the minimum requirement for RC responsiveness
- No blocking delays were observed
- Frequency variations were minimal

This confirms deterministic execution behavior.

7.4. End-to-End Command Latency

End-to-end latency was measured from:

- User input on the mobile device
- To observable actuator response

Latency components include:

- Bluetooth transmission delay
- Packet parsing delay
- Control loop synchronization delay

7.4.1. Observed Behavior

Testing indicates:

- No perceptible delay in manual control
- Immediate motor and servo reaction
- Stable control during rapid command changes

Measured latency remained within acceptable bounds for real-time manual control applications.

7.5. Actuator Response Stability

PWM signals were verified for:

- Consistent duty cycle generation
- Stable servo pulse width
- Absence of jitter-induced irregularities

Hardware-assisted PWM ensured stable actuator control independent of minor loop timing variations.

7.6. Wireless Communication Stability

Long-duration tests were performed to evaluate:

- Packet loss occurrence
- Connection drop frequency
- Recovery behavior

Results show:

- Stable Bluetooth connectivity within expected operating range
- Proper handling of packet validation
- No firmware crashes during communication stress

7.7. Telemetry Throughput

Telemetry packet transmission rate was evaluated to ensure it does not interfere with control responsiveness.

Observations include:

- Telemetry packets transmitted without saturating communication channel
- Control commands remain prioritized
- No buffer overflows detected

This confirms appropriate bandwidth allocation.

7.8. Memory Utilization

Memory usage was analyzed based on:

- Static memory allocation
- Buffer sizes
- Stack usage

Because dynamic allocation is avoided:

- Memory usage remains predictable
- Fragmentation risks are eliminated

The firmware operates well within the available memory limits of the microcontroller.

7.9. CPU Utilization Estimation

CPU utilization can be approximated as:

$$U = \frac{T_{loop}}{T_{available}}$$

Given the high processing capability of the microcontroller relative to the task complexity:

- CPU load remains low
- Significant computational margin exists
- Additional features can be integrated without compromising stability

7.10. Stress Testing

Stress tests included:

- Rapid command changes
- Continuous telemetry transmission
- Simultaneous motor and servo operation

The system maintained:

- Stable loop frequency
- No watchdog resets
- No observable performance degradation

7.11. Performance Limitations

Identified performance constraints include:

- Bluetooth bandwidth limitations
- I2C transaction latency
- Physical actuator response time

These limitations are external to core firmware execution and do not indicate architectural flaws.

7.12. Summary

Experimental evaluation confirms:

- Deterministic control loop execution
- Acceptable end-to-end latency
- Stable actuator control
- Reliable wireless communication
- Low resource utilization

The measured performance aligns with the theoretical analysis presented in Chapter 5 and validates the effectiveness of the modular firmware architecture.

Capítulo 8

Design Decisions and Trade-Off Analysis

8.1. Introduction

This chapter analyzes the principal design decisions taken during the development of the system and evaluates the associated trade-offs.

Rather than merely describing implementation details, this chapter focuses on the reasoning process behind architectural, hardware, and firmware choices.

The discussion covers:

- Microcontroller selection
- Communication protocol design
- Modular firmware structure
- Hardware abstraction strategy
- Real-time execution model
- Expansion methodology

8.2. Microcontroller Selection

The selection of the ESP32 microcontroller was based on several factors.

8.2.1. Advantages

- Integrated Bluetooth connectivity
- Sufficient computational resources
- Hardware PWM and multiple communication peripherals
- Low cost relative to capabilities

8.2.2. Trade-Offs

- Increased architectural complexity compared to simpler microcontrollers
- Higher power consumption than minimal 8-bit devices
- More complex development environment

Despite these trade-offs, the integration of wireless capability justified its selection, reducing overall system complexity.

8.3. Custom Communication Protocol

A custom binary protocol was designed instead of using standardized text-based formats.

8.3.1. Advantages

- Lower bandwidth usage
- Reduced parsing overhead
- Deterministic packet size
- Explicit checksum validation

8.3.2. Trade-Offs

- Increased development effort
- Reduced human readability
- Requires custom debugging tools

The performance and determinism benefits outweighed the additional development complexity.

8.4. Modular Firmware Architecture

The firmware was structured into independent modules.

8.4.1. Advantages

- Improved maintainability
- Clear separation of responsibilities
- Simplified testing
- Scalability support

8.4.2. Trade-Offs

- Slightly increased structural complexity
- More header files and interfaces to manage

The modular design ensures long-term extensibility and code clarity.

8.5. Hardware Abstraction Layer

Hardware abstraction was implemented to isolate peripheral-specific details.

8.5.1. Advantages

- Simplified hardware replacement
- Reduced coupling between logic and electrical configuration
- Improved portability

8.5.2. Trade-Offs

- Additional wrapper functions
- Slight performance overhead

Given the processing margin available, abstraction overhead is negligible compared to architectural clarity.

8.6. Single-Threaded Execution Model

The system uses a cooperative single-loop execution model rather than an RTOS.

8.6.1. Advantages

- Simpler debugging
- Predictable execution order
- Reduced synchronization complexity

8.6.2. Trade-Offs

- No true parallel task execution
- Timing analysis must ensure non-blocking behavior

For soft real-time RC control, this model provides sufficient determinism without unnecessary complexity.

8.7. I2C-Based GPIO Expansion

External GPIO expansion was implemented using an I2C interface.

8.7.1. Advantages

- Efficient pin utilization
- Modular hardware growth
- Reduced PCB complexity

8.7.2. Trade-Offs

- Increased communication latency
- Dependency on shared bus integrity

The flexibility gained from expandable I/O justifies the minor timing overhead.

8.8. Static Memory Allocation Strategy

Dynamic memory allocation was intentionally avoided.

8.8.1. Advantages

- Deterministic memory usage
- Elimination of fragmentation risk
- Simplified reliability analysis

8.8.2. Trade-Offs

- Fixed buffer sizes
- Reduced runtime flexibility

Predictability was prioritized over dynamic adaptability due to real-time constraints.

8.9. Binary Packet Structure

Fixed-length packet structures were selected instead of variable-length formats.

8.9.1. Advantages

- Simplified parsing logic
- Reduced runtime validation complexity
- Deterministic buffer management

8.9.2. Trade-Offs

- Potential underutilization of packet payload space
- Reduced flexibility for heterogeneous message types

For control systems, determinism was considered more valuable than format flexibility.

8.10. System Scalability Philosophy

The system was designed with forward scalability in mind.

Design considerations included:

- Modular firmware layers
- Expandable hardware interfaces
- Clearly defined communication structure

This prevents architectural redesign when integrating new features.

8.11. Overall Architectural Philosophy

The overarching design philosophy prioritized:

- Determinism over complexity
- Modularity over monolithic optimization
- Predictability over maximal performance
- Expandability over minimal footprint

These priorities align with engineering best practices for embedded control systems.

8.12. Summary

The system design reflects deliberate engineering trade-offs balancing:

- Performance
- Complexity
- Expandability
- Maintainability
- Determinism

Each decision was evaluated not only for immediate functionality but also for long-term system robustness and scalability.

Capítulo 9

Future Work

9.1. Introduction

Although the implemented system satisfies its functional and performance objectives, multiple opportunities exist for further improvement and expansion.

This chapter outlines potential future developments that could enhance system capability, robustness, scalability, and technological sophistication.

9.2. Closed-Loop Control Implementation

The current system operates primarily under open-loop control for motor speed and steering position.

Future work may include:

- Integration of wheel encoders
- Implementation of PID speed control
- Closed-loop steering angle correction

Closed-loop control would improve:

- Motion accuracy
- Load compensation
- Terrain adaptability

9.3. Migration to Real-Time Operating System

While the cooperative loop model has proven sufficient, migration to an RTOS could provide:

- Task prioritization
- Independent timing domains
- Improved scalability

- Deterministic multi-task scheduling

An RTOS-based implementation would be especially beneficial if the system integrates:

- Video streaming
- Advanced sensor fusion
- Autonomous navigation algorithms

9.4. Advanced Communication Protocol

Enhancements to the custom communication protocol may include:

- Packet sequencing and acknowledgment
- Encryption for secure communication
- Adaptive data rate control
- Error correction mechanisms

These additions would improve reliability in high-interference environments.

9.5. Wi-Fi and Internet Connectivity

Expanding beyond Bluetooth communication could enable:

- Remote control over Wi-Fi
- Cloud telemetry logging
- Over-the-air firmware updates
- Multi-user control sessions

Internet connectivity would transform the system from a local controller into a distributed control platform.

9.6. Energy Optimization

Future revisions may include:

- Dynamic frequency scaling
- Sleep mode implementation
- Battery monitoring and management system
- Power-efficient communication scheduling

Energy optimization would extend operational time in battery-powered deployments.

9.7. Autonomous Operation Capabilities

The current system focuses on manual control.

Future work may incorporate:

- Obstacle detection
- Path planning algorithms
- GPS-based navigation
- Sensor fusion frameworks

This would enable partial or fully autonomous operation modes.

9.8. Improved Telemetry and Data Analytics

Telemetry functionality could be extended to include:

- Real-time performance logging
- Data visualization dashboards
- Long-term storage for performance analysis
- Predictive maintenance indicators

Data-driven analysis would enhance system monitoring and diagnostics.

9.9. Custom PCB Design

The current implementation may use development boards and modular drivers.

Future hardware iterations could include:

- Fully integrated custom PCB
- Optimized trace routing
- Reduced physical footprint
- Improved EMI shielding

A custom PCB would improve mechanical robustness and production readiness.

9.10. Mechanical System Integration

Further integration between firmware and mechanical systems may include:

- Suspension feedback sensors
- Load measurement systems
- Torque monitoring

Tighter integration would allow advanced vehicle dynamics control.

9.11. Scalability to Multi-Vehicle Systems

The architecture could be extended to support:

- Multiple vehicles under centralized control
- Swarm coordination algorithms
- Distributed telemetry aggregation

This would require protocol redesign and communication arbitration mechanisms.

9.12. Artificial Intelligence Integration

Future versions may integrate:

- Machine learning for adaptive control
- Computer vision modules
- Behavioral learning systems

These features would significantly increase system autonomy and complexity.

9.13. Summary

The presented system establishes a modular and scalable embedded control platform.

Future developments may enhance:

- Control precision
- Communication robustness
- Energy efficiency
- Autonomy level
- Hardware integration

The current architecture was intentionally designed to accommodate such expansions without requiring fundamental redesign.

This confirms that the project serves not only as a functional implementation but also as a foundation for continued technological advancement.